

Knight Ascendent 2

엔진: Unity 6000.0.49f1

장르: 3D, 액션, 소울라이크, 무기 수집형 MMORPG

개발기간: 2025.04.07 ~ 2025.08.22

GitHub 주소 : (클라이언트) <https://github.com/Nerbe72/ProjectA2>

(서버) <https://github.com/Nerbe72/ProjectA2-Server>

Youtube 주소: <https://youtu.be/uc4My2V9fzc>



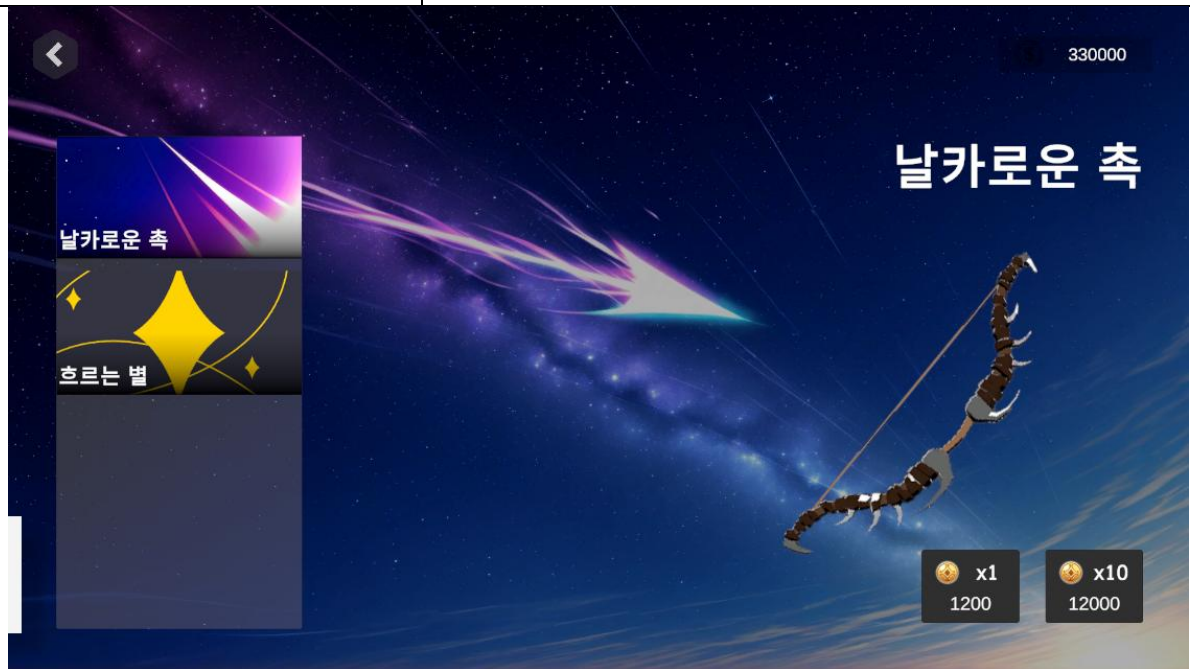
설명

소울라이크 RPG 를 기반으로 하며 수집형 게임의 핵심 시스템인 뽑기 시스템을 무기 아이템과 접목시켰습니다. 멀티 플레이를 지원하며 몬스터와 접속중인 모든 플레이어간 상호작용이 가능합니다. 이외에도 강화, 스킬 장비 귀속 시스템 등 다양한 콘텐츠를 포함시켰습니다.

개발 목표

1. 수집형 가차 시스템 및 node 콘텐츠 서버
2. json
3. photon 서버 동기화
4. CSV 데이터, 로케일 테이블
5. 아이템 관리, 정렬, 필터 및 이터레이터 패턴
6. 아이템 제작, 강화, 스킬귀속 및 어댑터 패턴, 팩토리 패턴
7. 퀘스트 콘텐츠 및 path
8. NPC 대화 다이얼로그, 레벨업
9. IK 를 활용한 적응형 사운드
10. 이벤트 처리 및 옵저버 패턴]
11. 창 관리, 싱글톤 관리
12. 비동기 로드
13. 행동트리 / 유한상태기계

기술 요약	가차 시스템과 node 콘텐츠 서버
-------	---------------------



```

"banners": [
  {
    "BannerType": "Pickup",
    "BannerName": 1000000001,
    "SSR_PickupList": [10004],
    "SR_PickupList": [10001, 10006],
    "SSR_Percent": 10,
    "SR_Percent": 19,
    "SSR_PickupPercent": 100,
    "CharacterPosition": { "x": 0, "y": 0 },
    "BannerPath": "bow_banner",
    "BackgroundPath": "bow_bg",
    "SinglePrice": 1200,
    "TenPrice": 12000
  },
  { "CharacterId": 10002, "Damage": 109, "Defense": 91, "TotalCount": 100, "SRCurrentCount": 1, "SSRCurrentCount": 1, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:0" },
  { "CharacterId": 10003, "Damage": 307, "Defense": 185, "TotalCount": 161, "SRCurrentCount": 2, "SSRCurrentCount": 2, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:0" },
  { "CharacterId": 10003, "Damage": 339, "Defense": 109, "TotalCount": 162, "SRCurrentCount": 3, "SSRCurrentCount": 3, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:1" },
  { "CharacterId": 10005, "Damage": 163, "Defense": 84, "TotalCount": 163, "SRCurrentCount": 4, "SSRCurrentCount": 4, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:2" },
  { "CharacterId": 10005, "Damage": 157, "Defense": 90, "TotalCount": 164, "SRCurrentCount": 5, "SSRCurrentCount": 5, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:3" },
  { "CharacterId": 10002, "Damage": 236, "Defense": 93, "TotalCount": 165, "SRCurrentCount": 6, "SSRCurrentCount": 6, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:4" },
  { "CharacterId": 10005, "Damage": 183, "Defense": 84, "TotalCount": 166, "SRCurrentCount": 7, "SSRCurrentCount": 7, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:5" },
  { "CharacterId": 10002, "Damage": 218, "Defense": 64, "TotalCount": 167, "SRCurrentCount": 8, "SSRCurrentCount": 8, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:6" },
  { "CharacterId": 10003, "Damage": 416, "Defense": 192, "TotalCount": 168, "SRCurrentCount": 9, "SSRCurrentCount": 9, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:7" },
  { "CharacterId": 10006, "Damage": 691, "Defense": 218, "TotalCount": 169, "SRCurrentCount": 0, "SSRCurrentCount": 10, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:8" },
  { "CharacterId": 10005, "Damage": 218, "Defense": 89, "TotalCount": 170, "SRCurrentCount": 1, "SSRCurrentCount": 11, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-27-23-47-27:9" },
  { "CharacterId": 10003, "Damage": 351, "Defense": 161, "TotalCount": 171, "SRCurrentCount": 2, "SSRCurrentCount": 12, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:0" },
  { "CharacterId": 10002, "Damage": 247, "Defense": 81, "TotalCount": 172, "SRCurrentCount": 3, "SSRCurrentCount": 13, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:1" },
  { "CharacterId": 10001, "Damage": 320, "Defense": 75, "TotalCount": 173, "SRCurrentCount": 0, "SSRCurrentCount": 14, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:2" },
  { "CharacterId": 10006, "Damage": 629, "Defense": 270, "TotalCount": 174, "SRCurrentCount": 0, "SSRCurrentCount": 15, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:3" },
  { "CharacterId": 10003, "Damage": 444, "Defense": 211, "TotalCount": 175, "SRCurrentCount": 1, "SSRCurrentCount": 16, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:4" },
  { "CharacterId": 10005, "Damage": 127, "Defense": 90, "TotalCount": 176, "SRCurrentCount": 2, "SSRCurrentCount": 17, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:5" },
  { "CharacterId": 10002, "Damage": 159, "Defense": 92, "TotalCount": 177, "SRCurrentCount": 3, "SSRCurrentCount": 18, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:6" },
  { "CharacterId": 10002, "Damage": 152, "Defense": 71, "TotalCount": 178, "SRCurrentCount": 4, "SSRCurrentCount": 19, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:7" },
  { "CharacterId": 10005, "Damage": 133, "Defense": 83, "TotalCount": 179, "SRCurrentCount": 5, "SSRCurrentCount": 20, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:8" },
  { "CharacterId": 10006, "Damage": 612, "Defense": 241, "TotalCount": 180, "SRCurrentCount": 0, "SSRCurrentCount": 21, "PickupForce": false, "Time": "(UTC+09:00) 2025-8-28-2-16-2:9" }
]

```

가차에 필요한 배너와 가차 기록 데이터는 매 가차 페이지를 여는 순간 로드하게 됩니다.

가차 기록 데이터로는
 등장한 무기 ID,
 무기의 랜덤 설정된 기본스탯,
 천장 시스템을 위한 각 등급별 스택과 전체 스택,
 반천장 시스템을 위한 강제픽업 bool,
 그리고 가차를 수행한 시간
 으로 구성되어 있습니다.

가차 시스템은 기본적으로 1 회, 10 회로 구별되어 있습니다.



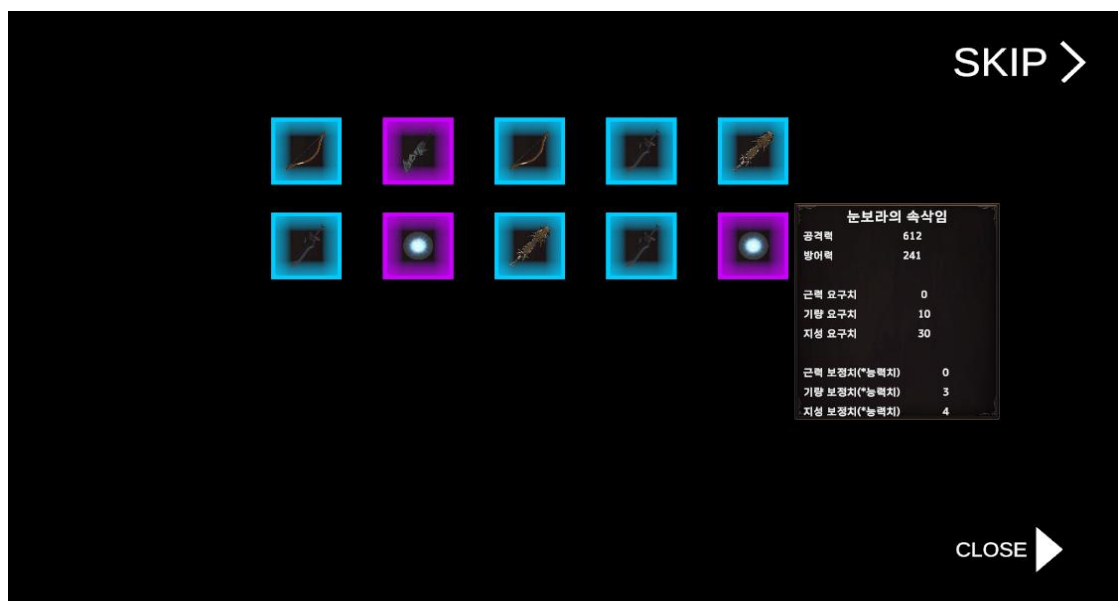
10 회의 경우 미니게임을 수행하게 됩니다.

미니게임은 AssetBundle 을 통해 불러오도록 구성하였습니다. 따라서 추후 에셋 번들만 배포하여 미니게임을 추가로 제공할 수 도 있습니다.

각 확률은 배너 데이터에 설정된 %값을 따르며,

미니게임을 성공하면 SSR 등장 확률에 고정된 5%p 를 추가합니다.

미니게임은 10 회뽑기시 30% 확률도 등장하도록 설계했지만 현재는 100% 확률로 등장합니다.



이후 개별 결과창과 최종 결과창을 볼 수 있으며 커서를 호버하여 정보를 볼 수 있습니다.



가차의 경우 미니게임이 완료된 직후 랜덤으로 무기를 생성하며, 무기가 무사히 인벤토리에 저장된 시점에서 결과창을 표시합니다.

따라서 결과창을 표시하는 도중에 게임이 종료되어도 데이터가 결손되지 않습니다.

Nodejs 서버

노드 서버에서는 모든 데이터를 json 의 형태로 저장 및 요청하여 총 세 가지 기능을 수행합니다.

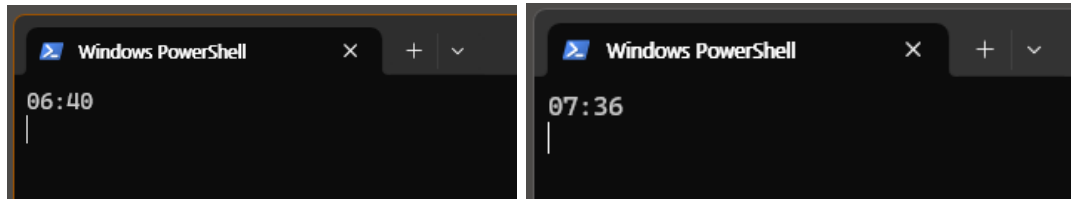
로그인

클라이언트로 부터 로그인 요청을 받으면 DB.json 파일을 읽고 id,pw 의 매칭을 검사합니다.

매칭되는 데이터가 있다면 jwt 토큰을 발급하고 추가로 uid 와 username 을 실어 클라이언트로 성공 여부를 전송합니다.

추후 이 token 을 유저 검증용 미들웨어 토큰으로 사용합니다.

시간 동기화



서버는 켜진 순간부터 시간이 흐릅니다. 유니티의 각도 90 도 = 12 시, 270 도 = 00 시로 세팅 후 이를 기반으로 서버의 시간을 0~360 까지 순환합니다.

속도는 초당 1 도씩 변경되며 매 초 인게임 4 분입니다.

클라이언트에서는 코루틴을 통해 1 초 간격으로 서버로부터 값을 받아옵니다.

검증단계를 없애고 요청하기 때문에 누구나 요청 가능하며 클라이언트에서는 요청된 값을 다음 요청까지 대기하는 1 초간 이전 회전값으로부터 lerp 합니다.

가차 배너정보

클라이언트에서 플레이어가 가차 페이지를 열면 배너 데이터를 banners 엔드포인트로 요청하며, 서버에서는 별도의 후처리 없이 json 을 파싱하여 텍스트로 받아옵니다,

텍스트는 FromJson 을 통해 제네릭 타입으로 json 을 다시 해당 타입으로 역직렬화 합니다.

이후 클라이언트는 역직렬화된 배너 데이터를 이용해 가차 페이지 내의 배너 리스트를 초기화하고 후속 작업을 진행합니다.

가차 로그

```
{"CharacterId":10004,"Damage":331,"Defense":147,"TotalCount":90,"SRCurrentCount":0,"SSRCurrentCount":0,"PickupForce":false,"Time":"(UTC+09:00) 2025-9-8-20-47-1:9"}
```

가차 로그는 위 와 같은 json 데이터를 개행으로 구분된 형태로 저장되어 있습니다.

각 데이터는 가차 1 회에 해당됩니다.

요청

플레이어가 가차 페이지를 열면 토큰을 실어 readgachalog 엔드포인트로 로그 데이터를 요청합니다.

서버에서는 습득한 토큰을 검증하고 획득한 uid 를 이용해 로그가 저장된 폴더를 찾습니다.

파일이 없다면 빈 배열을 반환하고 파일이 있다면 가차 기록 전체를 리스트의 형태로 전달합니다.

가차 결과의 가장 마지막 데이터를 이용해 현재 가차 스택을 불러오고 세팅한 후 가차를 수행합니다.

저장

클라이언트에서 가차를 수행하면 결과값과 스택, 시간을 포함한 클래스를 생성합니다.

이 클래스 리스트를 보유한 GachaResultWrapper 를, 서버의 writegachalog 엔드포인트에 전달합니다.

서버는 토큰을 통해 uid 를 획득하고 파일이 있는지 체크한 뒤 전달받은 데이터를 파일의 끝에 붙여넣습니다.

기술 요약	Photon 서버 동기화
	플레이어가 로그인을 수행하면 포톤 서버에 접속합니다. 접속 후 즉시 로비로 이동하며 확실히 접속되었음을 보장하기 위해 로비의 입장을 대기합니다. 만약 로비 입장에 실패하면 게임 진입 버튼이 활성화되지 않습니다. 게임에 진입하면 저장된 데이터로부터 맵 ID 를 불러와 해당 맵으로 플레이어를 이동시킵니다. 이 때 포톤의 룸에 입장하게 되며 이 역시 룸에 입장하기를 대기합니다. 룸에 진입한 후에 맵을 로드합니다. 맵 로드시에는 <code>messageQueueRunning</code> 를 비활성화 했다가 맵이 로드가 끝난 직후 다시 활성화하여 로딩 중의 불완전한 포톤 데이터 수신을 방지했습니다. 씬을 로드하는 과정에서 포톤의 관심그룹을 설정합니다. 현재 맵에 있지 않은 오브젝트나 대상을 로드할 가능성을 배제하기 위해 이동하려는 맵의 ID 를 <code>byte</code> 로 캐스팅하여 관심그룹 ID 로 설정합니다. 마스터 클라이언트가 된 클라이언트는 몬스터의 생명 주기를 담당합니다. 마스터 클라이언트는 테이블로부터 불러온 데이터를 기반으로 해당 맵에 배치되어야 할 몬스터를 포톤 오브젝트로 생성합니다. 플레이어의 위치, 회전, 애니메이션을 포톤을 통해 동기화 시켰습니다. 플레이어의 위치와 회전값은 <code>IpunObservable</code> 을 이용해 스트림 하도록 했습니다. 해당 인터페이스의 함수 내에서 호출되는 <code>stream</code> 은 자동으로 제어 대상은 쓰기 모드로 설정되므로 별도의 사전 설정 또한 필요하지 않습니다. 플레이어는 각자 <code>position</code>, <code>rotation</code>, <code>linearVelocity</code> 를 주고받습니다. 이 때 플레이어의 자연스러운 위치 이동을 보여주기 위해 현재 시각-메시지 전송 시간을 뺀 값을 <code>lag</code> 으로 지칭, $networkPosition = networkPosition + (linearVelocity * lag)$으로 속도 기반 보정을 수행한 뒤 <code>networkPosition</code> 으로 <code>MoveTowards</code> 시켜 점프를 포함하여 자연스러운 이동 보간을 구현했습니다. 애니메이션의 경우에는 포톤의 내장된 기능을 이용했습니다.



플레이어의 무기를 동기화 시킵니다. 플레이어가 무기를 장착하면 장착된 무기의 ID 를 RPC 를 통해 전달하며 대상이 되는 오브젝트에 무기를 장착시킵니다.

무기 장착시 isMine 이 아닌 player 는 무기의 collider 를 제거하여 적이 중복으로 공격되는 문제를 차단했습니다.

적은 마스터 클라이언트에 의해 스폰됩니다.

적 또한 플레이어와 동일한 방식으로 IpunObservable 을 이용해 이동을 동기화하고 PhotonView 를 통해 애니메이션을 동기화 합니다.

플레이어와의 상호작용의 경우 피격과 사망을 RPC 를 이용해 동기화합니다.

모든 적은 피격시 RPC 를 호출하여 마스터 클라이언트로 피격된 공격 타입과 데미지를 전달하며 마스터 클라이언트는 이 정보를 이용해 적의 정보를 갱신하고

사망 상태를 체크하여 다시 모든 클라이언트에게 피격 혹은 사망 상태를 RPC 를 통해 모든 플레이어에게 전달합니다.

사망시에는 사망 시킨 플레이어의 actorNumber 를 함께 전달하며,

해당 actorNumber 와 일치하는 player 에게만 재화가 지급되며 해당 클라이언트에서만 아이템을 드롭하게 됩니다.

기술 요약	데이터 테이블
-------	---------

Locale_ID	KO	EN	JA	Item_ID	Name	Icon	Item_Type	Rarity	Prefab	Description
0	클릭하여	Click to Enter		1002	10001002	Deadly_Pc		1	2	11001002
1	게임종료	Exit Game		1004	10001004	Magic_Bol		1	3	11001004
4	마을 앞	Front Village		10001	10010001	Demon_Sv		0	2	Demon_Sv 11010001
5	마을	Village		10002	10010002	Sword_7		0	1	Sword_7 11010002
6	던전	Dungeon		10003	10010003	Wooden_E		0	1	Wooden_E 11010003
100	레어도	Rarity		10004	10010004	Goblin_Bo		0	3	Goblin_Bo 11010004
101	데미지	Damage		10005	10010005	Teeth		0	1	Teeth 11010005
102	방어	Defense		10006	10010006	Whisper_B		0	2	Whisper_B 11010006
110	전체	All		10007	10010007	Karnea_As		0	3	Karnea_As 11010007
111	무기	Weapon								
112	포션	Potion								
200	스크롤 레어	Scroll Rarity		60001	10060001	Scroll_Tee		2	1	11060001

테이블은 Document 폴더에 .csv 의 형식으로 저장되며 빌드 이전에 바이너리 파일로 변환시켜 사용합니다.

테이블에는 아이템 정보, 적의 스폰 위치, UI 및 아이템 텍스트 로케일과 대화 로케일 등의 정보를 개별 테이블로 저장합니다.

아이템의 경우에는 무기 테이블/스킬 스크롤 ID 와 연결됩니다.

Weapon_ID	Weapon_Type	Attack_Type	Damage_Min	Damage_Max	Defense_Min	Defense_Max	Require_S	Require_D	Require_In	Damage_C	Damage_C	Damage_C	CastingTime	Projectile_Amount	Enchantment	AttackSound
10001	0	0	200	320	50	100	10	5	0	4	2	0	0	0	2	15 Melee1
10002	0	0	140	260	50	100	0	0	0	3	0	0	0	0	1	15 Melee1
10003	1	0	300	450	100	220	8	10	0	2	4	0	0	11002	1	15 Bow1
10004	1	0	300	440	120	230	5	15	0	2	3	0	0	11002	3	10 Bow1
10005	0	2	100	230	80	90	3	3	0	2	2	2	0.9	0	2	15 Melee2
10006	2	1	550	710	200	300	0	10	30	0	3	4	1	11005	2	15 Magic1
10007	2	1	480	800	180	250	0	8	35	0	2	6	0	0	3	10 Magic1
90001	1	0	0	0	0	0	0	0	0	0	0	0	0	91001	0	0 Bow1
90002	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0 Melee2

무기 테이블에는 가차의 랜덤 값 지정을 위한 공격력과 방어력 최대 최소범위와 무기의 요구스탯, 공격력 보정치, 그리고 스킬을 장착할 수 있는 슬롯 개수와 강화 최대치 등의 정보를 보유하고 있습니다.

ID	CreationTime	Price	Ingred1	Ingred2	Ingred3	Ingred4	Ingred5	Ingred6	Ingred7	Ingred8	Ingred9	Ingred10
60001	10005	100	70001:4	70002:2	e							
60002	1002	1000	70002:10	70003:4	e							
60004	1004	2500	70002:14	70004:12	e							

스킬 스크롤의 경우에는 제작될 아이템과 제작비용, 이후 e 가 등장하기 전까지 탐색하며 재료의 종류와 개수를 가져옵니다.

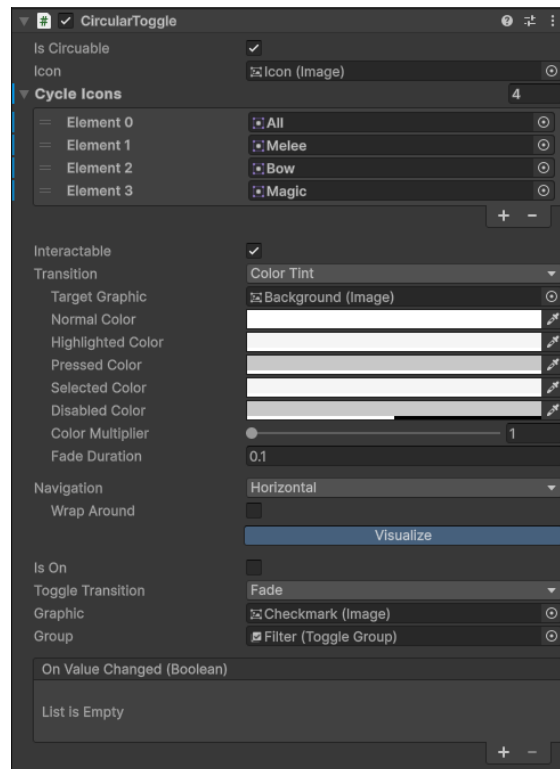
NPCID	Priority	Condition	Value	ToDialogID
90000001	1	2	1 1000003	5
90000001	2	2	1 1000004	8

이외에도 단순히 불러오는 것이 아닌 튜플을 키 값으로 사용하는 NPC 대화 컨디션 분기 테이블 등 다양한 형태로 테이블이 관리됩니다.

기술 요약	커스텀토글, 이터레이터(인벤토리, 필터)
-------	------------------------



필터는 기본적으로 프리팹으로 제공되며 이 필터를 통해 이벤트를 구독받아 리스트의 결과값을 수정하게 됩니다.

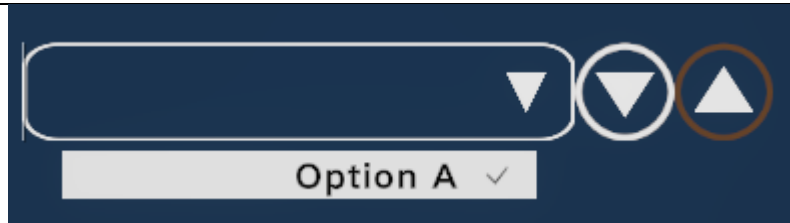


아이템 토글 중 특히 무기 토글의 경우 여러 번 누르면 해당 토글 내에서도 전체, 근거리, 원거리 등으로 순환하며 필터링 됩니다.

이런 기능을 토글 자체를 이용해 제어하기에는 어려움이 있어 커스텀 토글을 제작했습니다.

기본적으로 Iscircuable 이 켜져있다면 대상이 되는 Image 와 순환할 아이콘 이미지를 제공받아 cycle icons 의 개수만큼 순환하게 됩니다.

순환토글은 EventTrigger 컴포넌트를 활용하여 키 입력을 감지하기 때문에 해당 컴포넌트가 강제됩니다. 토글의 순환은 Filter 에서 제어되며 순환시 선택된 인덱스에 맞춰 아이콘이 변화합니다.



정렬의 경우 정렬 타입을 위한 드롭다운과 우측의 오름차순 내림차순 버튼이 있습니다.

사용되는 각 위치에서 초기화와 사용을 직접 수행하며 enum 을 통해 고정된 타입을 명시하여 사용합니다.

```
/// <summary>
/// +100 로케일
/// </summary>
참조 28개
public enum SortType
{
    Rarity = 0,
    Damage = 1,
    Defense = 2,
}
```

각 옵션의 언어 또한 enum 의 값에 100 을 더한 값을 로케일의 키 값으로 지정하여 다양한 언어를 지원할 수 있도록 디테일을 추가했습니다.

```
"InventoryIDString": "3db65543-1ea2-4cce-97d0-aca443933899",
"ItemID": 10003,
"Damage": 432,
"Defense": 172
```

현재 보유중인 아이템은 리스트로 저장됩니다. 하지만 아이템 프레임은 고유한 인벤토리 ID 값(Guid)을 이용하여 딕셔너리로 저장합니다.

아이템 리스트는 이터레이터 패턴을 이용해 순차 탐색하여 해당하는 아이템을 탐색하게 되며,

아이템 프레임 딕셔너리는 선택한 프레임의 ID 값을 이용해 해당 ID 를 보유한 아이템을 가져오기 위해 사용됩니다.

기술 요약	어댑터, 팩토리(강화, 스킬, 제작)
-------	----------------------

```
{
  "weaponInventoryIDString": "06ecc2ae-6ac0-4dab-b372-c28d8655e8dc",
  "enhancedLevel": 5,
  "enchantedSkillIds": [
    1002
  ]
},
```

무기의 강화정보와 스킬 귀속 정보는 어댑터 패턴을 이용해 무기와 연결시켰습니다.

파라미터를 추가하기엔 어렵기 때문에 확장성이 있도록 인벤토리 id 를 중심으로 아이템과 weaponEnhancementAdapter 를 참조하도록 구성하였습니다.

해당 어댑터는 인벤토리에서 Guid 를 키로 한 딕셔너리의 형태로 저장되며 아이템으로 부터 어댑터를, 어댑터로부터 아이템을 상호 참조할 수 있도록 함수를 구성하였습니다.



WeaponID	Enhance	Add_STR	Add_DEX	Add_INT	Growth
10001	0	0	0	0	0
10002	0	0	0	0	0
10002	1	0	1	0	0
10002	2	0	2	0	0
10002	3	1	2	0	0
10002	4	1	3	0	0
10002	5	1	4	0	0
10002	6	1	5	0	0
10002	7	2	5	0	0
10002	8	2	6	0	0
10002	9	2	7	0	0
10002	10	2	8	0	0
10002	11	3	8	0	0
10002	12	3	9	0	0
10002	13	3	10	0	0
10002	14	3	11	0	0
10002	15	4	12	0	0

어댑터를 통해 무기를 강화할 수 있습니다.

강화의 경우 무기 테이블로부터 최대 강화수치를 가져와 해당 수치만큼 강화를 수행하게 됩니다.

강화가 된 무기는 어댑터 내의 강화 수치를 보유하게 되며, 아이템 ID 와 강화수치를 이용해 추가 보정치를 테이블로부터 가져옵니다.



어댑터를 통해 스킬을 귀속시킬 수 있습니다.

무기는 종류별로 고유의 스킬 슬롯이 존재하며, 스킬 또한 고유의 소모 슬롯을 보유하고 있습니다.

귀속 시도시에 현재 무기의 어댑터를 참고하여 장착중인 스킬 목록을 탐색하며,

테이블을 통해 슬롯 개수에서 스킬들의 슬롯을 뺍니다.

장착시킬 슬롯 개수만큼 슬롯의 여유가 있다면 어댑터에 스킬 id 를 기록합니다.

이렇게 어댑터를 활용하여 무기의 강화, 귀속 시스템을 제작하여 폐쇄된 구조에 기능을 추가하여 확장성을 확보했습니다.



아이템은 가차, 드롭 아이템 획득, 퀘스트 클리어를 통해 획득 가능합니다.

여러가지 획득처와 여러 아이템 타입을 대응하고 한곳에서 관리할 수 있도록 아이템 팩토리를 사용하였습니다.

획득된 아이템 id 를 통해 테이블 정보를 가져옵니다.

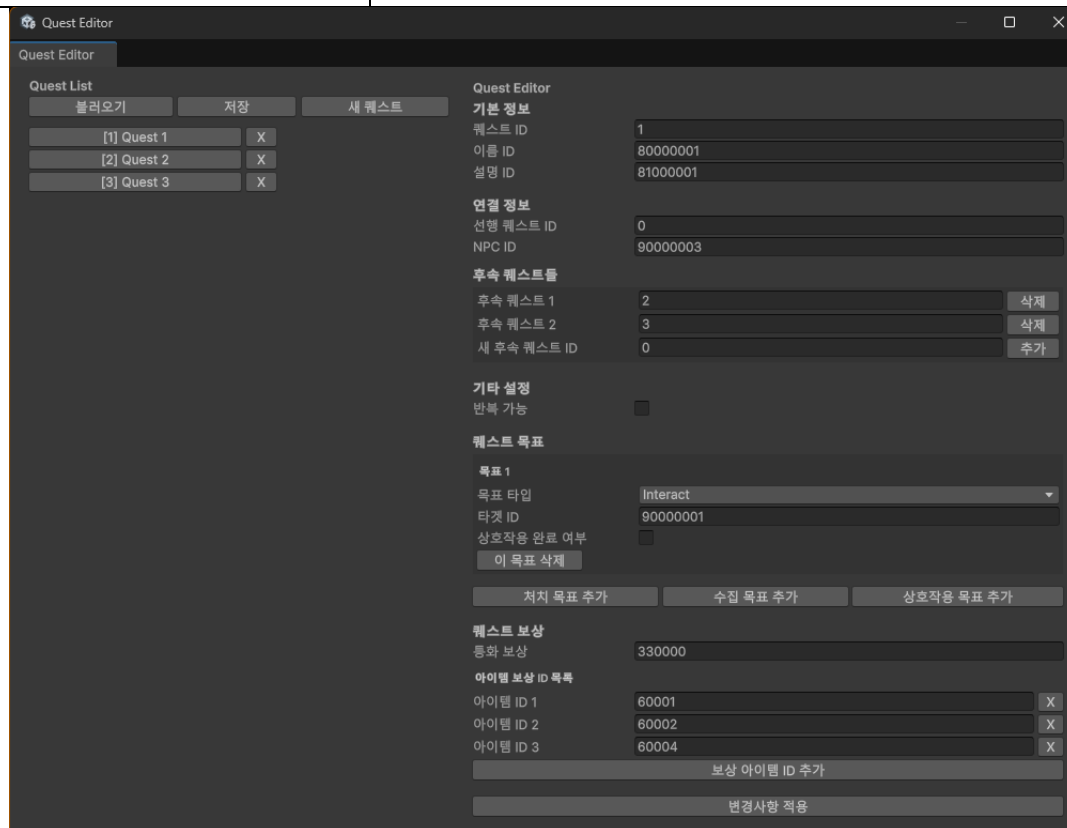
해당 테이블에서 아이템 타입을 가져온 뒤 해당 타입에 맞는 아이템을 생성하게 됩니다.

가차의 경우 조금 다른 노선을 타게 됩니다.

이미 가차 진행 과정에서 랜덤 수치를 부여 받았기 때문에 해당 정보를 포함하여 무기 아이템을 생성하게 됩니다.

무기의 경우 인벤토리에 저장하는 과정에서 어댑터를 함께 생성하여 저장합니다.

기술 요약	퀘스트, path
-------	-----------



퀘스트는 손쉽게 추가, 제거, 수정할 수 있도록 유니티 에디터창을 이용해서 퀘스트 에디터를 제작했습니다.

```
"questList": [
  {
    "QuestID": 1,
    "NameID": 80000001,
    "DescriptionID": 81000001,
    "Type": 0,
    "PrerequisiteQuestID": 0,
    "NextQuestIDs": [
      2,
      3
    ],
    "ReceiverNPCID": 90000003,
    "Repeatable": false,
    "Objectives": [
      {
        "ObjectiveIndex": 0,
        "TargetID": 90000001,
        "Required": 0,
        "Interacted": false,
        "ObjectiveType": 0
      }
    ],
    "Reward": {
      "Currency": 330000,
      "itemIds": [
        60001,
        60002,
        60004
      ]
    }
  },
],
```

저장하면 json 의 형태로 저장되며 로그인시 node 서버에서 불러와 QuestManager 에 프리로드됩니다.

퀘스트는 여러 개의 목표를 가질 수 있으며,

이 목표는 타겟과 대화/타겟 사냥/아이템 보유. 세 가지 타입이 있습니다.

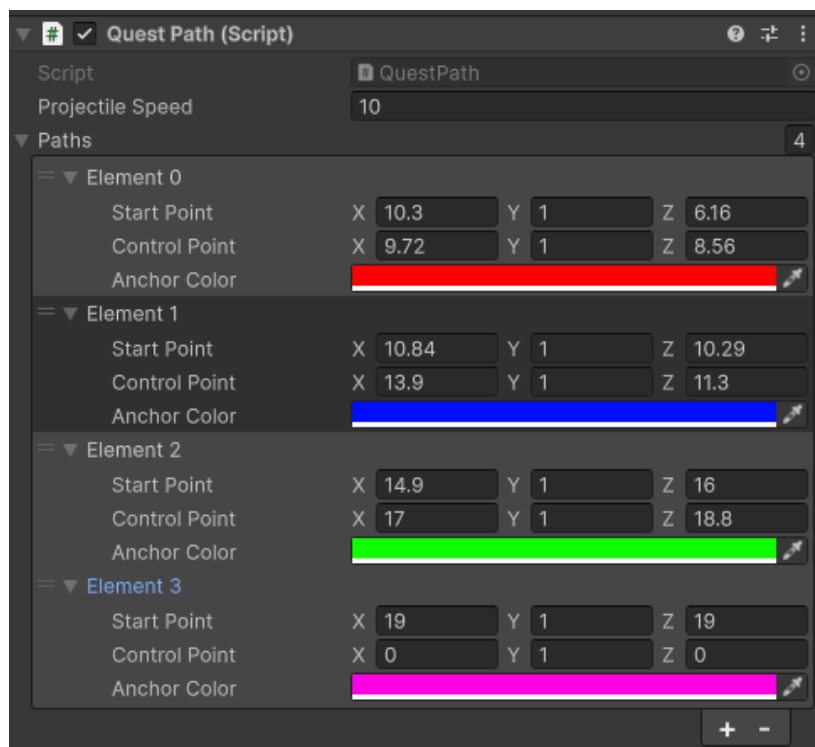
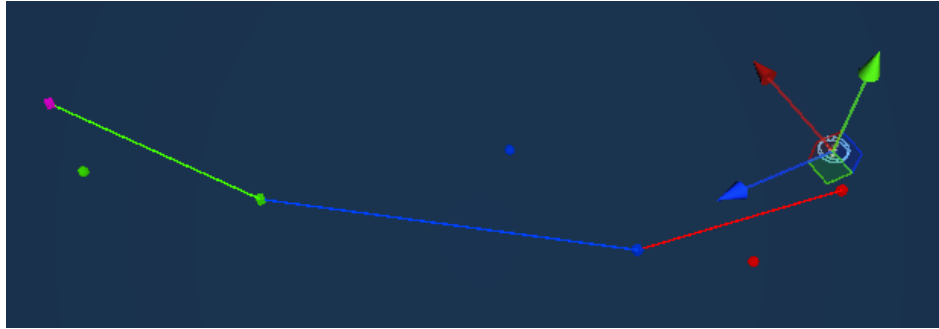
모든 목표를 수행하면 퀘스트 달성으로 기록되며 대상에게 다시 돌아가 말을 걸면 퀘스트가 완료됩니다.

퀘스트는 후속 퀘스트가 존재하며 퀘스트 매니저 내에서 트리의 형태로 저장됩니다.

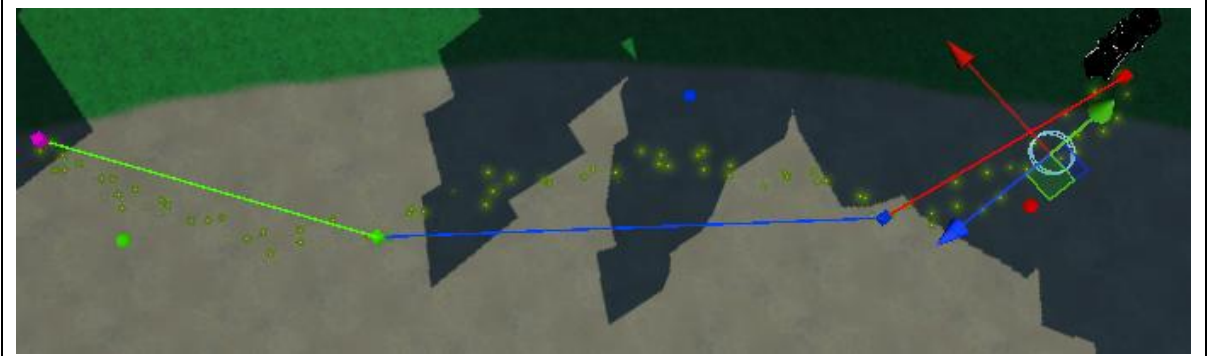
이는 퀘스트 목록에서 깊이우선탐색을 수행하며 정렬합니다.

퀘스트 완수시 아이템이나 재화 등을 보상으로 획득할 수 있습니다. 아이템은 id 기반으로, 팩토리를 통해 생성된 후 인벤토리에 저장됩니다.

편의를 위해 퀘스트 완료 아이템은 30 개씩 획득하도록 고정되어 있습니다.



퀘스트는 path 프리팹을 이용해 경로를 표시합니다.



Path 를 위한 커스텀 컴포넌트를 제작하였으며 컨트롤 포인트에 의해 배지어 곡선으로 경로를 따라 파티클이 이동합니다

경로의 끝에 도달하면 처음 위치부터 다시 이동합니다

기술 요약	IK (발걸음 사운드)
	Animator 의 IK 기능을 이용해 발이 지면을 관통하는 것을 막고 추가로 발소리를 구현했습니다. 애니메이션 이벤트를 이용하여 설정할 수도 있지만 이는 모든 동작에 대응하기 힘들기 때문에 IK 를 이용하면 모든 애니메이션에 대응할 수 있다는 장점이 있습니다. IK 의 왼발과 오른발의 위치로부터 ray 를 쏘아 지면을 감지하고, 충돌 위치로 발을 고정하고 발의 회전값을 지면의 법선벡터를 이용해 지면에 달라붙도록 합니다. 또한 해당 지면의 레이어에 맞춰 발자국 소리가 동적으로 변경됩니다.

기술 요약	옵저버 패턴 (레벨업, 스탯)
플레이어는 레벨업 혹은 무기 장착 등 스탯에 변화를 주는 행동을 수행하면 그 정보에 맞춰 모든 UI가 갱신됩니다. 매 프레임 갱신되었음을 감시하면 게임의 성능에 영향을 줄 뿐만 아니라 중복된 호출부에 대한 유지보수가 어렵습니다. 따라서 Action을 통해 옵저버 패턴의 형태로 레벨업과 무기 장착 등의 동작을 수행한 시점에 Invoke만 수행하여 수정된 값을 반영합니다. 또한 언어 변경의 경우에도 localeID값만 고정하고 Locale enum의 변화만 감지하여 UI내의 모든 텍스트를 그에 맞는 언어로 변경하도록 했습니다.	

기술 요약	WindowStack / Singleton
WindowStack 플레이어 혹은 NPC 가 여는 UI 창을 자동으로 관리하도록 구성하기 위해 IWindowStack 인터페이스를 제작했습니다. 인터페이스가 포함된 WindowBase 를 기반으로 WindowStackMnager 내에서 창의 열고 닫힘을 관리합니다. 모든 창은 WindowBase 를 기본적으로 상속받습니다. 특정 키 입력혹은 대화창을 통해 창이 열리면 매니저에 스택으로 자신을 추가합니다. 매니저는 스택에 쌓인 창이 위에 보이도록 창의 SortingOrder 를 (stackCount+10) 으로 변경합니다. 이는 AddWindow 시 자동으로 수행됩니다. 창을 닫기 위해 ESC 키를 입력하는 순간에는 가장 위 스택부터 순차로 닫게 됩니다. 이는 열린 창의 역순으로 자연스럽게 닫히도록 유도합니다.	

Singleton

만약 유저가 로비로 이동하게 된다면 불필요한 오브젝트와 매니저에 저장된 데이터를 모두 초기화 시켜줘야 합니다.

또한 싱글톤을 등록시키고 static 으로 한곳에서 저장하는 클래스가 코드적으로 비대해지고 매번 추가해줘야 하는 불편함이 있었습니다.

이를 해결하기 위해 싱글톤을 아래와 같이 딕셔너리의 형태로 저장했습니다.

```
Dictionary<System.Type, (int priority, UnityEngine.MonoBehaviour data)>
```

또한 Add, Get 함수를 만들어 제네릭 타입으로 저장 및 요청할 수 있도록 했습니다.

Add 시에는 추가된 순서대로 자동으로 priority(우선순위)가 설정됩니다.

Get 시에는 typeof(T)를 저장하는 키 값을 이용해 O(1)의 시간복잡도로 요청할 수 있도록 설계했습니다.

유저가 인게임에서 벗어나 로비로 이동하게 되면 저장된 모든 싱글톤 오브젝트를 파괴합니다.

이 때 priority 를 이용해 정렬한 뒤 역순으로 해제시킵니다. 이후 마지막으로 플레이어와 인벤토리를 삭제합니다.

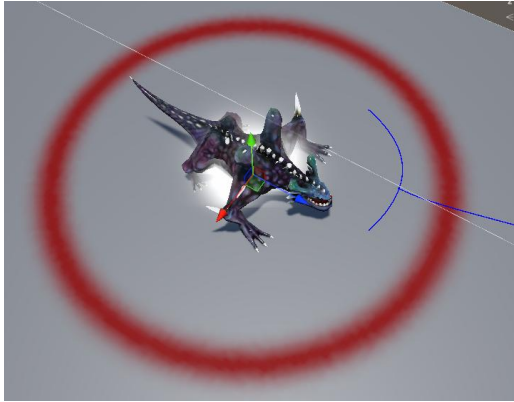
싱글톤을 참조하는 싱글톤의 존재 가능성을 고려하여 첫 로딩시에 전, 후로 나뉘어 싱글톤이 생성되기 때문에 역순 해제시 자연스럽게 오류 없이 해제시킬 수 있습니다.

기술 요약	비동기
무기 생성 무기의 오브젝트 프리팹을 로드할 때 비동기를 사용합니다. 다만 무기를 장착할 때 오브젝트가 로드되지 않은 경우 장착이 무시되거나 오류가 발생할 가능성을 최소화 시켰습니다. 무기를 로드하는 시점은 무기가 인벤토리에 보유된 경우입니다. 이는 세이브 데이터로부터 로드될 때를 포함합니다. 이렇게 로드 시점을 장착시가 아닌 아이템 획득시로 바꾸어 장착하기 까지의 시간을 자연스럽게 확보하고 이 시간동안 로드되도록 유도했습니다. 이로써 메모리 사용을 최소화 하면서도 끊김없이 무기를 자연스럽게 로드하고 무기 장착시 정상적으로 보일 수 있도록 했습니다.	
서버 요청 서버로부터 로그인하거나 데이터를 요청하는 경우에도 비동기로 로드합니다. 외부로부터 데이터를 가져오는 경우 반드시 비동기를 통해 로드해야 게임의 끊김과 멈춤을 줄일 수 있습니다. 다만 데이터를 획득하는 것을 완수해야하기에 데이터가 정상적으로 전달될 때 까지 대기 후 동작을 수행합니다. 로그인의 경우 로그인이 완료된 시점까지 대기한 후 후속 작업을 진행하도록 설계했습니다.	
가차 가차의 모든 동작은 비동기로 수행됩니다. 가차 수행시 발생하는 미니게임을 대기하는 것 역시 비동기로 대기 후 플래그를 통해 다음으로 넘깁니다. 이후 아이템의 instance 를 비동기로 생성하고 생성이 완료될 때 까지 대기합니다. 생성이 완료되고 인벤토리에 저장이 완료되면 splash, single, total 결과 페이지를 순차로 실행합니다. 이 경우에도 각 페이지가 완료되거나 스킵될 때 까지의 플래그를 각자 설정하기까지 비동기의 형태로 대기하게 됩니다. 하나의 함수 혹은 과정 내에서 비동기를 수행하기 때문에 상태를 추적하기 용이한 장점이 있습니다.	

기술 요약	Behaviour Tree / FSM
행동트리 적과 보스는 행동트리를 기반으로 동작합니다. 행동트리는 사전에 스크립트의 형태로 구성된 노드와 이를 연결하는 CreateBehaviorTree 함수를 통해 트리를 초기화합니다. 고블린과 스켈레톤 등 일반 적은 시야 정보를 갖습니다. 해당 시야에 들어온 플레이어를 감지하거나 피격당한 경우 행동을 시작합니다. 가장 최우선 순위는 적의 사망 상태이며 이후 공격 거리를 판단하고 거리가 멀다면 플레이어를 향해 다가갑니다. 스켈레톤의 경우에는 너무 가까운 거리에 플레이어가 접근하면 공격하기 위한 거리를 벌리게 됩니다. 적들은 모두 플레이어가 시야 내에 없으며 스폰 위치로부터 정해진 거리를 벗어난 경우 스폰 위치로 귀환합니다. 보스의 경우 플레이어에 대한 시야를 가지지 않습니다. 마찬가지로 최우선 사항은 사망 상태이며, 시야 대신 조우 상태를 감지하게 됩니다. 조우 상태는 trigger 에 플레이어가 진입한 상태면 수행되는 컷씬이 종료된 시점에 true 가 되며 보스가 행동을 시작합니다. 마찬가지로 플레이어를 향해 접근하거나 혹은 patrol 하며 플레이어를 바라본 상태로 좌 우로 천천히 이동합니다. 이동 상태는 확률에 의해 동작하게 됩니다. (50% 접근, 12.5% 대기/후진/좌/우 이동)	

보스는 이동하는 시간이 정해져 있으며 해당 이동 패턴이 끝나면 공격을 수행합니다.

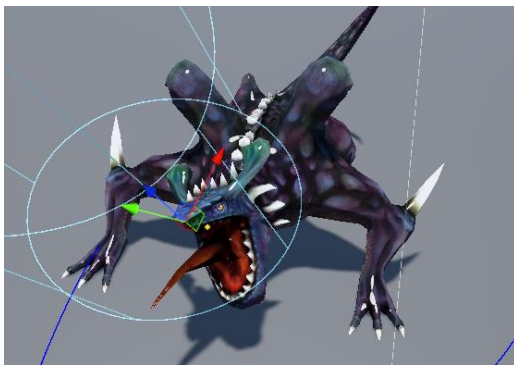
공격의 경우 플레이어와의 거리에 따라 n 개의 공격 타입을 구사합니다.



점프 지면 강타 후 충격파



클로 내려찍기



물어뜯기

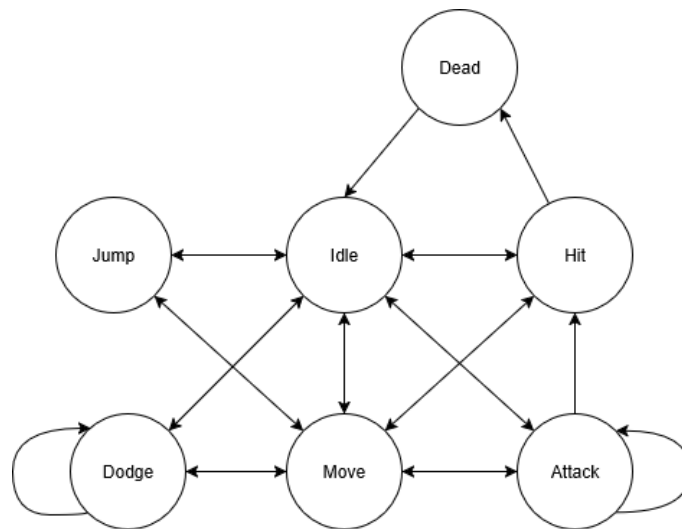


정면 박치기

Pattern Range	
▼ Pattern 1 Melee	
Min Distance	0
Max Distance	4.5
▼ Pattern 2 Mid Range	
Min Distance	4
Max Distance	5
▼ Pattern 3 Long Range	
Min Distance	4.8
Max Distance	7
▼ Pattern 4 Charge	
Min Distance	6
Max Distance	10

각 공격의 거리는 위와 같습니다.

유한상태기계



플레이어는 유한 상태 기계를 통해 각 동작 중 한가지 동작만 수행할 수 있도록 제한했습니다.

상태는 대기, 이동, 공격, 피격, 회피, 점프, 사망이 있습니다.

대기 상태에서는 이동 입력이 주어지면 이동 상태로 전이됩니다.

공격중에 피격되면 피격 상태로 전이되며 피격 상태에서는 애니메이션이 완료되기 전에는 다른 동작으로 전이 불가능 합니다. 단, 피격 상태에서 체력이 0 이 되면 사망 상태로 전이됩니다.

점프 상태에는 공격으로 전이가 불가능 하지만 느린 속도로 이동 가능하며 점프 종료시 키 입력 상태에 따라 대기 혹은 이동 상태로 전이됩니다.